RcppML: High-Performance Non-Negative Matrix Factorization with Cross-Validation for R

Zachary DeBruine
University of Michigan
debruine@umich.edu

Abstract

Non-negative matrix factorization (NMF) is a fundamental dimensionality reduction technique with applications spanning bioinformatics, text mining, computer vision, and signal processing. This paper presents **RcppML**, an R package that implements a comprehensive NMF framework with several key innovations: (1) a highly optimized sequential coordinate descent solver for non-negative least squares with support for L1 regularization and bounded solutions; (2) streaming cross-validation with minimal computational overhead using a novel position-dependent random number generator; (3) multiple robust loss functions including MSE, MAE, Huber, and KL-divergence implemented via iteratively reweighted least squares; (4) automatic rank determination through golden section search optimization; (5) specialized rank-2 NMF for high-performance spectral bipartitioning. The package provides unified support for both sparse and dense matrices through template metaprogramming and achieves state-of-the-art performance through careful cache-aware algorithm design.

Keywords: non-negative matrix factorization, cross-validation, coordinate descent, spectral clustering, R, C++, **Rcpp**

1 Introduction

Non-negative matrix factorization (NMF) decomposes a non-negative matrix $\mathbf{A} \in \mathbb{R}_{\geq 0}^{m \times n}$ into two non-negative factor matrices $\mathbf{W} \in \mathbb{R}_{\geq 0}^{m \times k}$ and $\mathbf{H} \in \mathbb{R}_{\geq 0}^{k \times n}$ such that $\mathbf{A} \approx \mathbf{WH}$ (Lee and Seung, 1999). The non-negativity constraints enable parts-based, interpretable representations that have proven valuable

across diverse domains including facial recognition, text mining, gene expression analysis (Brunet et al., 2004), and single-cell genomics.

The **RcppML** package for R addresses several practical challenges:

1. **Integrated cross-validation**: A streaming algorithm that computes held-out reconstruction error with minimal overhead (<5% typical).
2. **Robust loss functions**: Support for MSE, MAE, Huber, and KL-divergence losses implemented via iteratively reweighted least squares (IRLS).
3. **Automatic rank determination**: Golden section search optimization over the cross-validation objective to automatically identify optimal rank.
4. **Specialized rank-2 NMF**: A highly optimized implementation for spectral bipartitioning that outperforms truncated SVD approaches.
5. **High-performance implementation**: Cache-aware sequential coordinate descent with unified support for sparse and dense matrices.

2 NMF Algorithm

2.1 Problem Formulation

We formulate NMF with a scaling diagonal:

$$\min_{\mathbf{W},\mathbf{D},\mathbf{H}} \mathcal{L}(\mathbf{A}, \mathbf{WDH}) + \lambda_1 \|\mathbf{W}\|_1 + \lambda_1 \|\mathbf{H}\|_1 \tag{1}$$

subject to $\mathbf{W}, \mathbf{H} \geq 0$, $\mathbf{D} = \text{diag}(d_1, \ldots, d_k)$, and optional column normalization $\|\mathbf{w}_j\|_2 = 1$.

2.2 Alternating Least Squares

We optimize using alternating least squares (ALS), iteratively solving:

$$\mathbf{H} \leftarrow \arg\min_{\mathbf{H}\geq 0} \|\mathbf{A} - \mathbf{WDH}\|_F^2 + \lambda_1 \|\mathbf{H}\|_1 \tag{2}$$

$$\mathbf{W} \leftarrow \arg\min_{\mathbf{W}\geq 0} \|\mathbf{A}^\top - \mathbf{H}^\top \mathbf{D} \mathbf{W}^\top\|_F^2 + \lambda_1 \|\mathbf{W}\|_1 \tag{3}$$

2.3 Sequential Coordinate Descent Solver

We solve NNLS using sequential coordinate descent (Franc et al., 2005):

$$h_i \leftarrow \max\left(0, \frac{\mathbf{w}_i^\top(\mathbf{a} - \tilde{\mathbf{W}}\mathbf{h}) + h_i\|\mathbf{w}_i\|_2^2 - \lambda_1}{\|\mathbf{w}_i\|_2^2}\right) \tag{4}$$

The key insight is to maintain the residual $\mathbf{r} = \mathbf{a} - \tilde{\mathbf{W}}\mathbf{h}$ incrementally, reducing the complexity of each coordinate update from $O(mk)$ to $O(m)$.

3 Cross-Validation Framework

3.1 Position-Dependent Random Masking

We use a position-dependent random number generator that deterministically determines whether each entry is masked based on its position (i, j):

$$\text{mask}(i, j) = \mathbf{1}[\text{hash}(i, j, \text{seed}) < p_{\text{cv}}] \tag{5}$$

This "speckled" RNG has several advantages: (1) no memory allocation for the mask; (2) the same entry maps to the same mask value throughout optimization; (3) reproducibility through the seed.

3.2 Streaming Loss Accumulation

During each ALS iteration, we accumulate test error "for free" within the NNLS solve by checking the mask and accumulating squared residuals. This streaming approach has $O(1)$ memory overhead.

4 Loss Functions

RcppML supports four loss functions via IRLS:

- **MSE**: $\rho(r) = r^2$, $w(r) = 1$
- **MAE**: $\rho(r) = |r|$, $w(r) = 1/(|r| + \varepsilon)$
- **Huber**: Interpolates between L2 and L1
- **KL-divergence**: For count data

Matrix size	Density	`bipartition()` (ms)	`irlba()` (ms)
1000×500	10%	17	25

Table 1: Runtime comparison for rank-2 factorization.

5 Automatic Rank Determination

When the rank is unknown, **RcppML** supports automatic selection via `k = "auto"`. This performs golden section search over the cross-validation objective:

$$k^* = \arg \min_{k \in [k_{\min}, k_{\max}]} \text{CV-MSE}(k) \tag{6}$$

6 Rank-2 NMF for Spectral Bipartitioning

The `bipartition()` function implements a highly optimized rank-2 NMF for spectral bipartitioning. Given $\mathbf{A} \in \mathbb{R}_{\geq 0}^{m \times n}$, it returns a vector $\mathbf{v} \in \{-1, +1\}^n$ indicating cluster membership.

Table 1 compares `bipartition()` to `irlba()`.

The `dclust()` function builds on `bipartition()` to perform divisive hierarchical clustering using modularity as the stopping criterion.

7 Illustrations

7.1 Cross-Validation for Rank Selection

Figure 1 shows typical cross-validation curves. The test MSE decreases as rank increases then increases (overfitting).

7.2 Regularization Effects

Figure 2 shows the trade-off between reconstruction error and factor sparsity under L1 regularization.

7.3 Scaling Behavior

Figure 3 shows runtime scaling for sparse and dense matrices.

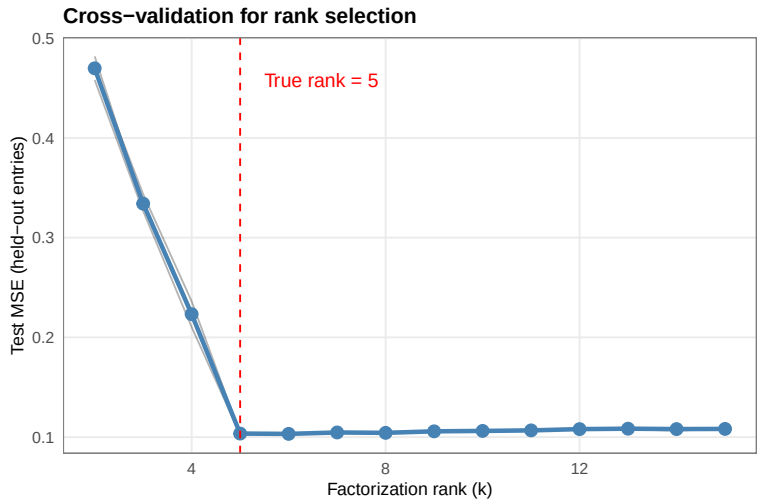

Figure 1: Cross-validation for rank selection on synthetic data with true rank 5. Test MSE is minimized near the true rank.

8 Summary

The **RcppML** package provides a comprehensive NMF implementation for R with integrated cross-validation, multiple loss functions via IRLS, automatic rank determination through golden section search, and specialized rank-2 NMF for spectral bipartitioning. The package is available on CRAN.

Computational Details

The results in this paper were obtained using R 4.5.0 with the packages **RcppML** 1.0.0, **Matrix** 1.7.3, **irlba** 2.3.5.1, and **ggplot2** 4.0.1.

References

- Brunet, J.-P., Tamayo, P., Golub, T. R., and Mesirov, J. P. (2004). Metagenes and molecular pattern discovery using matrix factorization. *PNAS*, 101(12):4164–4169.

- Eddelbuettel, D. and François, R. (2011). Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18.

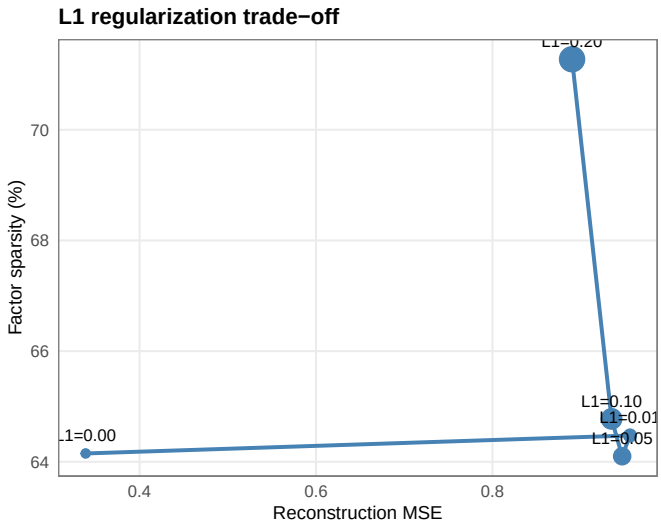

Figure 2: L1 regularization trade-off.

- Franc, V., Hlavac, V., and Navara, M. (2005). Sequential coordinate-wise algorithm for the non-negative least squares problem. *CAIP 2005*, pages 407–414.

- Lee, D. D. and Seung, H. S. (1999). Learning the parts of objects by non-negative matrix factorization. *Nature*, 401(6755):788–791.

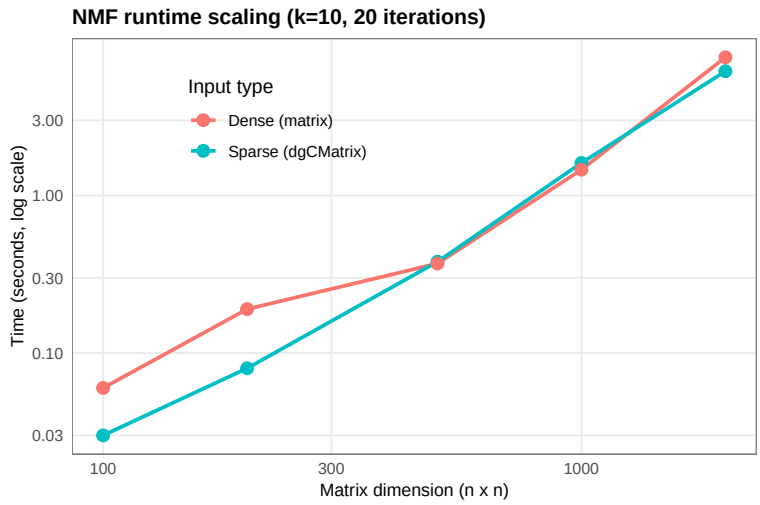

Figure 3: Runtime scaling for NMF on sparse and dense matrices.